

Project Proposal: Verifying a Basic Echo Server

Reagan Caliendo (rcali3) and Gianmarco Luciano (gluci3)

Project Overview:

The goal of this project is to implement a simple TCP-based echo server and formally verify its correctness properties. The server will receive messages from a client and return the same message back. Using formal verification techniques, we will prove key properties such as message integrity and memory safety.

Languages and Tools:

The program (client-server application) will be implemented in **C**, and the memory safety and correctness will be verified with **VeriFast**.

Top-Level Theorem Statement:

We aim to formally verify the following key properties of our echo server:

1. **Message Integrity:** The message received by the server is the same as the one sent by the client.
 - *Formal Theorem:* $\forall \text{msg}, \text{send}(\text{client}, \text{msg}) \rightarrow \text{receive}(\text{server}, \text{msg})$
 - This will ensure that the sent and received message are the same.
2. **No Buffer Overflow:** The program does not read or write beyond allocated buffer sizes.
 - *Formal Theorem:* $\forall i, 0 \leq i < \text{buffer_size} \rightarrow \text{buffer}[i] \text{ is valid}$
 - This will ensure that the program does not write beyond its buffer limit, which would result in security vulnerabilities.

Why This Project?

This project provides a practical application of formal verification to real-world network communication and we are both interested in network security so this is something relevant to our interests and anticipated career paths. It allows us to explore new tools other than Coq like VeriFast to ensure correctness and memory safety in C programs.

Possible Extensions:

If our project seems too simple and we are left with extra time, we have considered adding support for multiple clients and verifying order preservation of messages. This would be a similar approach but would ensure that the above proofs are valid in scenarios with multiple clients connected to a single server, for a more real world example.